

Esterian Conquest

Sysop Manual

Version 1.0.0-beta.1 — Beta

Copyright © 2026 Mason A. Green

Revision date: March 28, 2026



This manual © 2026 Mason A. Green and is licensed under CC BY-NC-SA 4.0.

License text: <https://creativecommons.org/licenses/by-nc-sa/4.0/>

Contents

1. Introduction	2
2. Getting Started: Hosting a Campaign	2
2.1. System Requirements	2
2.2. Building from Source	2
2.3. Directory Layout	2
2.4. Initializing a New Game	3
2.5. Recommended Hosted Play	3
3. Game Directory Structure	4
3.1. Core Files	4
3.2. Subdirectories	4
4. Configuration	5
4.0.1. Full Example	5
4.0.2. Stored Fields	5
4.0.3. session Block	6
4.0.4. inactivity Block	6
4.0.5. reservations Block	6
4.1. Environment Variables	6
5. Theming	7
5.1. Theme Selection	7
5.2. Player Theme Picker	7
5.3. Color Formats	7
5.4. Color Mode	8
5.5. Semantic Style Tokens	8
5.6. Mono Theme	9
6. SSH Access	10
7. Local and Direct Play	10
8. Legacy BBS Door Setup	10
8.1. Flags for Door Mode	10
8.2. Drop File	11
8.3. Enigma BBS (Rust Client)	11
9. File-Based Turn Submission	12
10. Turn Processing and Maintenance	12
11. Player Management	13
11.1. Reserving Seats	13
12. Classic Compatibility	14
13. Terminology	14
14. CLI Reference	14
14.1. ec-sysop	14
14.2. ec-game	15
15. Theme Token Reference	16

1. Introduction

Esterian Conquest (EC) is a multi-player strategy game originally written for DOS-era BBS systems. The Rust-native stack reimplements the game engine, player client, and admin tooling for modern systems while preserving classic compatibility at a well-defined import/export boundary.

This manual is for the **sysop** — the person responsible for hosting a game instance. It covers:

- installing and initializing a game
- configuration and theming
- recommended Nostr hosting with `ec-sysop nostr`
- localhost and direct deployment
- legacy BBS door compatibility
- player management
- turn processing and maintenance

For player-facing rules and gameplay, see the **Player Manual**.

This manual is the authoritative sysop manual for the Rust edition of Esterian Conquest. The preserved original `.DOC` set in `original/v1.5/` remains historical reference material and an ambiguity fallback for classic operator intent, not a higher-authority replacement for the current Rust manuals.

2. Getting Started: Hosting a Campaign

2.1. System Requirements

- Linux, macOS, or Windows
- Rust toolchain (stable) to build from source, or a pre-built binary release
- For the recommended public-hosted flow: an SSH-accessible host plus a Nostr relay reachable by players
- A terminal emulator for localhost or direct SSH play
- A BBS server with socket support only if you specifically want legacy door deployment

2.2. Building from Source

From the repository root:

```
cargo build --release -p ec-sysop -p ec-game -p ec-connect
```

The release binaries will be in `target/release/`.

2.3. Directory Layout

A running game occupies a single directory. A minimal populated game directory looks like:

```
/path/to/mygame/  
  ecgame.db           runtime database (SQLite)
```

All tools take `--dir /path/to/mygame` to locate the game.

2.4. Initializing a New Game

Create a new game with `ec-sysop new-game`:

```
ec-sysop new-game /srv/ec/games/friday-night --name "Friday Night EC" --players 4
```

This creates a fresh DB-only campaign directory containing exactly one runtime file: `ecgame.db`.

`ec-sysop new-game` does not generate classic `.DAT` files, does not copy the original DOS executables or `.DOC` manuals, and does not bootstrap `config.kdl` or a `themes/` subdirectory. Hosted Rust campaigns are SQLite-native. Classic import/export and oracle tooling remain available separately on the developer/compatibility boundary.

The supported public creation flags are:

Flag	Type	Description
<code>--name</code>	string	Optional human-readable game title stored in <code>ecgame.db</code> . If omitted, <code>ec-sysop</code> derives a title from the directory slug.
<code>--players</code>	integer	Number of empires. Supported range: 1–25. Defaults to 4.
<code>--year</code>	integer	Optional starting campaign year. Defaults to 3000.
<code>--seed</code>	integer	Optional integer seed for the campaign RNG. Controls map layout, starting positions, and all random events. If omitted, the engine picks a random seed and saves it to <code>ecgame.db</code> . The seed cannot be changed after creation.

NOTE: Use a different seed for every game. Reusing the same seed produces the same map, starting positions, and event sequence every time.

The target directory basename becomes the stable game slug. It must use only lowercase ASCII letters, digits, and dashes. The slug is distinct from the human-readable `game_name`, and both are distinct from the per-seat invite codes used by `ec-connect`.

2.5. Recommended Hosted Play

For new public campaigns, the recommended deployment path is `ec-sysop nostr` plus `ec-connect`. In that model, the sysop runs the normal Rust engine on a host, publishes the Nostr-facing daemon, and players join from their own machines with invite codes. The daemon handles identity and seat routing; the live game still runs inside `ec-game` over SSH. This keeps the original asynchronous EC rhythm without requiring per-player Unix account management or BBS door middleware.

A minimal hosted setup looks like:

```
ec-sysop new-game /srv/ec/games/friday-night --name "Friday Night EC" --players 4
ec-sysop host games add --config /etc/ec-gate/config.kdl --dir /srv/ec/games/friday-night
ec-sysop nostr init
ec-sysop nostr serve
```

After the daemon is running, view the hosted seat state and get the ready-to-distribute invite commands:

```
ec-sysop nostr seats --dir /path/to/mygame
```

The output lists every seat. Pending seats show two invite formats:

```
Seat 1 [pending]
  ec-connect --join ecinv1...
  amber-river@play.example.com --relay wss://relay.example.com --gate npub1...

Seat 2 [claimed]
  npub1...
```

Send each player his `ec-connect --join ecinv1...` line. The player:

1. Runs `ec-connect`.
2. Presses `N`.
3. Pastes the invite code and presses Enter.

That is the full player-side flow. The code is self-contained — it carries the relay URL, server host, and gate key. No extra configuration required.

On first join, `ec-connect` creates or unlocks the player's encrypted identity, claims the hosted seat through the relay, downloads the static starmap bundle, and opens the SSH-backed `ec-game` session. Returning players reconnect through `ec-connect` without re-entering any flags.

Power users and scripted workflows may also join directly from the command line:

```
ec-connect --join ecinv1...
```

Or with a plain two-word code (includes relay and gate flags for reliability):

```
ec-connect --join amber-river@play.example.com --relay wss://relay.example.com --gate npub1...
```

If an invite code is lost or compromised, reissue it:

```
ec-sysop nostr reissue --dir /path/to/mygame --player 2
```

This generates a fresh code for that seat, clears the old claim, and lets the player join again with the new code.

Hosted seat claims are stored in `ecgame.db`. That SQLite state is the authority for invite codes, claim status, and bound player `npubs`. Legacy `roster.kdl` files are migration input only.

3. Game Directory Structure

3.1. Core Files

ecgame.db The SQLite runtime database. All game state lives here. Do not edit manually; use `ec-sysop` for normal operator actions. Hosted Nostr seat claims also live here.

3.2. Subdirectories

exports/ Default root for classic `.DAT` export output. Can be overridden with `--export-root` or the `EC_CLIENT_EXPORT_ROOT` environment variable.

queue/ Default directory for turn order queue files. Can be overridden with `--queue-dir` or the `EC_CLIENT_QUEUE_DIR` environment variable.

4. Configuration

Hosted Rust campaigns do not use a per-game `config.kdl`. Sysop-editable runtime policy now lives in SQLite alongside the rest of the campaign state. Use `ec-sysop settings ...` to inspect or change it:

```
ec-sysop settings show --dir /srv/ec/games/friday-night
ec-sysop settings set --dir /srv/ec/games/friday-night --game-name "Friday Night EC"
ec-sysop settings reserve --dir /srv/ec/games/friday-night --player 1 --alias SYSOP
```

4.0.1. Full Example

```
slug=friday-night
game_name=Friday Night EC
default_theme_key=tokyo_night
snoop=true
session_max_idle_minutes=10
session_minimum_time_minutes=0
session_local_timeout=false
session_remote_timeout=true
inactivity_purge_after_turns=0
inactivity_autopilot_after_turns=0
maintenance_enabled=true
maintenance_interval_minutes=10080
maintenance_next_due_unix_seconds=1775347200
reservation seat=1 alias=SYSOP
reservation seat=2 alias=NightShade
```

4.0.2. Stored Fields

Field	Type	Default	Description
game_name	string	"Esterian Conquest"	Display name shown in the main menu header.
default_theme_key	string	"tokyo_night"	Bundled campaign theme key used as the default for local/SSH play.
snoop	bool	#true	Enable sysop snoop mode.
reservations	rows	(absent)	Optional BBS/dropfile seat reservations by caller alias.
maintenance_enabled	bool	#true	Whether maint-all should advance this game when it becomes due.

maintenance_interval_minutes	integer	10080	Maintenance cadence in minutes. 10080 = one week.
maintenance_next_due_unix_seconds	integer	(auto)	Next scheduled maintenance time as a Unix timestamp.

4.0.3. session Block

Field	Type	Default	Description
max_idle_minutes	integer	10	Minutes of inactivity before session timeout. Range: 0–120.
minimum_time_minutes	integer	0	Minimum session time guarantee in minutes. Range: 0–120.
local_timeout	bool	#false	Apply timeout to local (non-remote) sessions.
remote_timeout	bool	#true	Apply timeout to remote sessions.

4.0.4. inactivity Block

Field	Type	Default	Description
purge_after_turns	integer	0	Turns of inactivity before a player is purged. 0 = disabled. Range: 0–100.
autopilot_after_turns	integer	0	Turns of inactivity before autopilot engages. 0 = disabled. Range: 0–100.

4.0.5. reservations Block

Field	Type	Default	Description
seat player=<N> alias="NAME"	entry	(absent)	Reserve empire slot N for a BBS/dropfile caller alias. Alias matching is ASCII case-insensitive.

NOTE: ec-sysop settings import-kdl --dir <game_dir> still exists as a one-time migration tool for older beta campaigns that carried a per-game config.kdl. Normal hosted Rust campaigns do not depend on that file at runtime.

4.1. Environment Variables

Variable	Description
EC_CLIENT_EXPORT_ROOT	Override the default export root directory.
EC_CLIENT_QUEUE_DIR	Override the default turn queue directory.
COLORTERM	Color depth hint. truecolor or 24bit enables 24-bit color.

TERM	Terminal type. A value containing <code>256color</code> enables 256-color mode.
------	---

5. Theming

`ec-game` uses bundled built-in themes. The campaign default theme is stored by key in `ecgame.db`, while each player's local choice is stored there as a per-player preference. Hosted Rust campaigns do not require a per-game `themes/` directory.

5.1. Theme Selection

`ec-game` resolves the theme in this order:

1. If the player has a saved theme preference in `ecgame.db`, use that key.
2. Otherwise, use the campaign `default_theme_key` from `ecgame.db`.
3. If the stored key is missing or invalid, fall back to the bundled `tokyo_night` theme.

On parse error, the bundled default is used so a corrupted theme never prevents players from connecting.

5.2. Player Theme Picker

From the Main Menu and First Time Menu in local-terminal sessions, players can open `C>olor Theme` to preview and apply the bundled themes shipped with `ec-game`. The picker stays open after `Enter` so players can try several looks before returning to the menu with `q`.

The picker exposes the compiled-in theme set plus a synthetic `Mono` option, which applies a monochrome projection over the current theme for players who prefer a plain white-on-black display.

Joined players save their selected local theme immediately as a per-player preference in `ecgame.db`. A player choosing a theme from First Time Menu before fully joining uses it for that session, and the preference is saved when the join finishes successfully.

If a stored player theme key later points to a missing or invalid theme, `ec-game` falls back to `tokyo_night` automatically. If a color theme still cannot be materialized, `Mono` remains the safe last-resort display.

5.3. Color Formats

Three color formats are supported:

Format	Example	Notes
Named ANSI-16	" <code>bright_blue</code> "	Safe for all terminals including BBS/door clients. Recommended default.
256-color index	" <code>idx:208</code> "	Requires <code>--color-mode 256</code> or <code>truecolor</code> . Downgraded to nearest named color in 16-color mode.
24-bit hex RGB	" <code>#ff8800</code> "	Requires <code>--color-mode truecolor</code> . Downgraded gracefully in lower color modes.

`tokyo_night` is the default: a rich dark palette using 24-bit hex colors for modern SSH and local terminals, downgraded gracefully to 256-color or named ANSI-16 on terminals that do not support truecolor. `mag16` is the ANSI-16 native alternative: a restrained dark palette using only named 16-color values, safe for all terminal types including BBS door clients. The bundled theme set includes several other alternates. All hex colors degrade gracefully — they are automatically mapped to the nearest 256-color index or 16-color name when needed.

5.4. Color Mode

`ec-game` selects a color mode at startup:

Mode	Description
<code>ansi16</code>	Classic 16-color ANSI. Safe for BBS/door and all terminal types.
<code>256</code>	256-color xterm palette.
<code>truecolor</code>	24-bit RGB. Best for modern local or SSH terminals.
<code>auto</code>	Detected from <code>COLORTERM</code> and <code>TERM</code> environment variables.

Override with the `--color-mode` flag:

```
ec-game --dir /path/to/mygame --player 1 --color-mode truecolor
```

When `--encoding cp437` is specified (BBS door mode), `ec-game` defaults to `ansi16` regardless of the environment, unless `--color-mode` is set explicitly.

5.5. Semantic Style Tokens

All required style tokens:

Token	Purpose
<code>body</code>	Default body text.
<code>title</code>	Screen and section titles.
<code>shell_title</code>	Outer shell border title text such as the <code>EC CONNECT</code> frame title. This should normally use the same background as <code>body</code> with a more prominent foreground accent.
<code>shell_label</code>	Outer shell border identity label text such as the active alias or shortened <code>npub</code> shown on the right side of the <code>EC CONNECT</code> frame. This should normally use the same background as <code>body</code> with its own accent color distinct from <code>shell_title</code> .
<code>menu</code>	Menu item text.
<code>menu_hotkey</code>	Menu hotkey letters.
<code>prompt</code>	Input prompt text.
<code>prompt_angle_delimiter</code>	Angle prompt punctuation such as the <code>< ></code> around explicit quit/cancel tokens like <code><Q></code> .
<code>prompt_square_delimiter</code>	Square prompt punctuation such as the <code>[]</code> around defaults like <code>[03,03]</code> or yes/no defaults like <code>[Y]</code> .

prompt_hotkey	Prompt hotkey letters.
prompt_notice_action	Action-notice accent in prompts.
bright	Emphasized body text.
logo	Title logo decoration.
intro_accent	Intro screen accent color.
intro_tribute	Intro screen tribute line.
stardate_label	Stardate label.
stardate_week	Stardate week value.
stardate_year	Stardate year value.
error	Error messages.
notice	Warning / notice messages.
status_label	Status bar labels.
status_value	Status bar values.
table_chrome	Table borders and separators.
table_header	Table column headers.
table_body	Table body rows.
disabled_row	Disabled / unavailable table rows.
selected	Selected / highlighted row.
alert	High-priority alert text.
help_header	Help overlay section headers.
help_panel	Help overlay body text.
map_dot	Star map background dots.
map_crosshair	Star map crosshair / cursor.
map_center	Star map center marker.
quote	Quote / flavor text body.
quote_author	Quote attribution.
report_header	Report page headers.
indicator_on	Active indicator (e.g. status lit).
indicator_off	Inactive indicator.

The prompt delimiters are styled separately from the key text they contain. In command rails and inline prompts, the inner letters still use `prompt_hotkey`, while `< >` and `[ ]` come from their own delimiter styles. This lets a theme keep the command keys legible while giving quit markers and defaults their own visual distinction.

5.6. Mono Theme

Mono is one of the local `C>olor` Theme picker entries. It applies a monochrome projection over the active theme and can be selected and saved like any other local player theme preference.

In BBS door mode, monochrome output still comes from the separate `A>ansi color ON/OFF` toggle rather than the theme picker.

6. SSH Access

The recommended hosted path above already uses SSH under the hood: players enter through `ec-connect`, and the daemon opens a PTY running `ec-game`. You can also run `ec-game` over SSH directly when you want a private shared-host setup, manual debugging, or a simple trusted deployment without the Nostr invite flow.

`ec-game` runs cleanly over SSH. No special flags are required for modern terminal sessions.

Color mode is auto-detected from the environment:

- `COLORTERM=truecolor` → 24-bit RGB
- `TERM` containing `256color` → 256-color
- Otherwise → 16-color ANSI fallback

Force a specific mode with `--color-mode` if your SSH setup does not propagate `COLORTERM` reliably:

```
ec-game --dir /path/to/mygame --player 1 --color-mode 256
```

UTF-8 encoding (the default) is correct for SSH sessions on modern terminals. Use `--encoding cp437` only if you are proxying through a BBS or a CP437 terminal emulator over SSH.

7. Local and Direct Play

Localhost play remains a fully supported secondary mode for solo campaigns, hotseat sessions, and sysop testing. Run `ec-game` directly in your terminal:

```
ec-game --dir /path/to/mygame --player 1
```

Color mode and encoding default to `auto / utf8`, which is correct for modern terminal emulators. The client detects `COLORTERM` and `TERM` automatically.

8. Legacy BBS Door Setup

`ec-game` can still run as a BBS door. This path is preserved for classic-host compatibility, not as the primary recommendation for new public campaigns. In door mode, the client reads from and writes to a socket instead of the local TTY, using CP437 encoding and 16-color ANSI.

8.1. Flags for Door Mode

```
ec-game \  
  --dir /path/to/mygame \  
  --player <1-based index> \  
  --encoding cp437 \  
  --color-mode ansi16
```

`--encoding cp437` switches box-drawing characters and other extended characters to the CP437 code page, which is required for correct rendering in classic ANSI-aware BBS terminals (SyncTERM, NetRunner, etc.).

When `--encoding cp437` is active, `--color-mode` defaults to `ansi16` automatically. Override only if you know your BBS clients support a richer color depth.

8.2. Drop File

`ec-game` can read a BBS drop file directly with `--dropfile <path>`:

```
ec-game \  
  --dir /path/to/mygame \  
  --dropfile /path/to/D00R32.SYS
```

Supported drop file formats (auto-detected by filename, case-insensitive):

- D00R32.SYS — modern standard (Enigma, Mystic, Talisman, etc.)
- D00R.SYS — legacy, widest BBS software support
- CHAIN.TXT — WWIV format

The drop file supplies the player alias and session time limit. Explicit CLI flags always override drop file values. When `--dropfile` is given and `--encoding` is not, encoding defaults to `cp437` automatically.

`--timeout <minutes>` sets a session time limit independently of a drop file.

Reserve seats in `ecgame.db` when you want the caller alias to determine the empire automatically:

```
ec-sysop settings reserve --dir /path/to/mygame --player 1 --alias SYSOP  
ec-sysop settings reserve --dir /path/to/mygame --player 2 --alias NightShade
```

With that in place, a reserved caller can launch with `--dropfile` alone. If the caller alias is not reserved, `--player` is still required. If both `--player` and `--dropfile` are supplied for a reserved caller, they must agree on the same empire slot.

NOTE: The original DOS `ECGAME.EXE v1.5` expects a strict 32-line WWIV-style `CHAIN.TXT` drop file. Enigma BBS-generated `D00R.SYS` / `DORINFO` files will crash `ECGAME.EXE`. See `docs/sysop/enigma-bbs-setup.md` for the full legacy DOS door path if you need to host the original binary.

8.3. Enigma BBS (Rust Client)

The native Rust `ec-game` door is now verified on both Mystic and ENiGMA½. For ENiGMA, use the `abracadabra` module with `dropFileType: D00R32`, `io: stdio`, and `encoding: cp437`. Pass `--dir`, `--dropfile`, `--encoding cp437`, and `--color-mode ansi16` to the client, or use the helper wrapper at `tools/bbs/run_ec_rust.sh`. Use `--player` for unreserved callers, or reserve the alias in `ecgame.db` and let `--dropfile` resolve the seat.

If Enigma writes a `D00R32.SYS`, you can pass it directly:

```
ec-game \  
  --dir /path/to/mygame \  
  --dropfile /path/to/D00R32.SYS
```

For a fuller ENiGMA½ Rust-door setup, including a ready abracadabra configuration, see [docs/sysop/enigma-rust-setup.md](#).

In BBS door mode, the reliable control contract is:

- HJKL for movement
- ^U / ^D for paging
- Q or Esc for back/quit

Do not rely on arrows or PgUp / PgDn for normal play through BBS hosts.

9. File-Based Turn Submission

For localhost, shared-host, or custom-client workflows, players can submit orders by writing a KDL turn file and passing it to `ec-game submit-turn`. Use `--check` to validate without writing, then run without it to apply:

```
ec-game submit-turn --check --dir /path/to/mygame --player 1 --file /path/to/player1-  
turn.kdl  
ec-game submit-turn --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
```

The `--player` value must match the `turn player=...` header in the file. If any order in the file is invalid, the entire submission is rejected and nothing is written. This is a direct apply command, not a queue or upload inbox.

A minimal turn file:

```
turn player=1 year=3000  
tax rate=37
```

A fuller example:

```
turn player=1 year=3000  
  
tax rate=37  
  
planet record=16 {  
  build points=4 kind="scout"  
}  
  
fleet record=1 {  
  order speed=3 kind="scout_system" x=16 y=13  
}  
  
message to=2 subject="Border" body="Watching the north lane."
```

For the full node reference and schema, see [docs/sysop/turn-kdl.md](#).

10. Turn Processing and Maintenance

Run yearly maintenance with:

```
ec-sysop maint /path/to/mygame [turns]
ec-sysop maint-all [--config /etc/ec-gate/config.kdl]
```

`ec-sysop maint` advances the campaign in `ecgame.db`. EC does not schedule maintenance by itself. In a real deployment, invoke `ec-sysop maint` from your host scheduler or BBS tooling:

- a `systemd` timer
- `cron`
- a BBS event runner
- or manual `sysop` operation

For multi-game Nostr hosting, prefer a single global timer that runs `ec-sysop maint-all`. It reads the configured game directories from the gate config, skips games whose next due time has not arrived yet, and also skips games with live session leases so a player is never interrupted by maintenance.

Do not treat game settings as a scheduler. Campaign policy such as theming, snoop, and inactivity thresholds lives in `ecgame.db`; maintenance timing still belongs to the host.

11. Player Management

Inactive-player policy is configured in `ecgame.db` under the campaign settings block. The two public thresholds are:

- `purge_after_turns`
- `autopilot_after_turns`

These values are runtime policy, not setup-time game creation input.

11.1. Reserving Seats

To reserve empire slots for specific BBS users:

```
ec-sysop settings reserve --dir /path/to/mygame --player 1 --alias SYSOP
ec-sysop settings reserve --dir /path/to/mygame --player 2 --alias NightShade
```

Each `seat` entry binds one 1-based empire slot to one caller alias. Alias matching is ASCII case-insensitive, so `NightShade`, `nightshade`, and `NIGHTSHADE` are treated as the same reservation.

With reservations in place, launch `ec-game` with `--dropfile` and let the caller alias choose the empire automatically:

```
ec-game --dir /path/to/mygame --dropfile /path/to/D00R32.SYS
```

Important rules:

- if the caller alias is reserved, `--player` becomes optional
- if the caller alias is not reserved, `--player` is still required
- if both `--player` and `--dropfile` are supplied for a reserved caller, they must match
- if a reservation conflicts with an already-stored different player handle, `ec-game` will stop with a clear error so the `sysop` can reconcile the reservation and the runtime state

Reserving a seat does not by itself join or pre-name the empire. It only routes the caller to the intended slot. The usual first-time join flow still claims an open empire, and that first successful join records the caller alias into the player record for later logins.

12. Classic Compatibility

The Rust-native public deployment path is `ec-sysop nostr`, `ec-connect`, and `ec-game`.

Classic .DAT import/export, oracle runs against the original binaries, and other preservation workflows still exist, but they belong to the internal `ec-cli` developer/compatibility surface rather than the normal `sysop` manual.

13. Terminology

game directory The directory containing `ecgame.db` and any lazily-created export or queue subdirectories for a single running game instance. Passed to all tools via `--dir`.

ec-sysop The public Rust command-line `sysop` tool for campaign creation maintenance, and Nostr hosting.

ec-connect The beta-quality player-side connection client for the recommended hosted flow. It manages the player's Nostr identity, joins games by invite code, downloads the static starmap bundle on first join, and opens the SSH-backed `ec-game` session.

ec-game The Rust TUI player client.

ecgame.db The SQLite database that is the runtime source of truth for the Rust engine.

campaign settings The `sysop`-managed runtime policy rows stored in `ecgame.db`. They control game name, snoop mode, theme key, seat reservations, maintenance cadence, and inactivity thresholds.

14. CLI Reference

14.1. ec-sysop

`ec-sysop <subcommand> [options]`

Subcommand	Purpose
<code>new-game</code>	Create a fresh DB-only campaign directory. Public flags: <code>--name</code> , <code>--players</code> , <code>--year</code> , and <code>--seed</code> .
<code>settings show set reserve unreserve</code>	Inspect or edit the per-game runtime policy stored in <code>ecgame.db</code> .
<code>host games list add remove</code>	Inspect or edit the global game registry in <code>/etc/ec-gate/config.kdl</code> .
<code>host status</code>	Summarize the configured host, served game directories, claim counts, busy state, and maintenance-due state.

<code>nostr init</code>	Initialize the Nostr-hosting identity and config for the recommended public multiplayer path.
<code>nostr serve</code>	Run the Nostr-facing daemon that authenticates players and launches <code>ec-game</code> sessions.
<code>nostr seats</code>	List the hosted seat state stored in <code>ecgame.db</code> for one game directory.
<code>nostr reissue</code>	Generate a fresh invite code for one hosted seat and clear its old <code>npub</code> claim.
<code>nostr migrate-roster</code>	Import a legacy <code>roster.kdl</code> into <code>ecgame.db</code> , copy its display name into the campaign settings rows, and archive the old roster file.
<code>maint</code>	Run one or more maintenance turns against <code>ecgame.db</code> .
<code>maint-all</code>	Sweep every game registered in the gate config, skipping games that are not due or that currently have active sessions.

14.2. ec-game

`ec-game --dir <game_dir> [--player <1-based index>] [options]`

`ec-game submit-turn [--check] --dir <game_dir> --player <record> --file <turn.kdl>`

Interactive client flags:

Flag	Description
<code>--dir <path></code>	Game directory containing <code>ecgame.db</code> . Required.
<code>--player <N></code>	1-based empire index. Required unless a reserved dropfile alias resolves the seat from <code>ecgame.db</code> campaign settings.
<code>--encoding <utf8 cp437></code>	Output encoding. Default: <code>utf8</code> . Use <code>cp437</code> for BBS/door mode.
<code>--color-mode <ansi16 256 truecolor auto></code>	Color depth. Default: <code>auto</code> (env-detected). CP437 mode defaults to <code>ansi16</code> .
<code>--dropfile <path></code>	Parse a BBS drop file (DOOR32.SYS, DOOR.SYS, or CHAIN.TXT). Supplies alias and timeout, defaults encoding to <code>cp437</code> , and can resolve the player seat through <code>ecgame.db</code> reservations. Explicit flags always override except that <code>--player</code> must match a reserved alias when both are present.

<code>--session-token <hex></code>	Hosted-session lease token injected by ec-gate during Nostr/SSH login. Normal local and BBS launches do not pass this flag.
<code>--timeout <minutes></code>	Session time limit in minutes. Overrides any drop file value.
<code>--export-root <path></code>	Override export directory. Default: <code><game_dir>/exports</code> .
<code>--queue-dir <path></code>	Override turn queue directory. Default: <code><game_dir>/queue</code> .

submit-turn flags:

Flag	Description
<code>--check</code>	Validate the KDL file without mutating the campaign.
<code>--dir <path></code>	Game directory containing <code>ecgame.db</code> . Required.
<code>--player <N></code>	1-based empire index. Required, and must match the KDL header.
<code>--file <path></code>	Turn submission KDL file to validate or apply. Required.

NOTE: `ec-game submit-turn` is all-or-nothing. If any command in the file is invalid, the entire submission is rejected and nothing is written to `ecgame.db`.

15. Theme Token Reference

The ANSI-compatible reference palette below matches the bundled `mag16` theme. The default campaign theme key is still `tokyo_night`, but `mag16` is a useful baseline because it shows the semantic token split using portable named ANSI colors.

Token	fg	bg	bold
body	white	black	—
title	bright_blue	black	yes
menu	white	black	—
menu_hotkey	yellow	black	yes
prompt	white	black	—
prompt_angle_delimiter	green	black	yes
prompt_square_delimiter	red	black	yes
prompt_hotkey	yellow	black	yes
prompt_notice_action	bright_cyan	black	yes
bright	bright_white	black	yes
logo	bright_blue	black	yes
intro_accent	bright_blue	black	—

intro_tribute	bright_magenta	black	—
stardate_label	cyan	black	—
stardate_week	bright_cyan	black	—
stardate_year	yellow	black	—
error	red	black	yes
notice	bright_red	black	yes
status_label	white	black	—
status_value	white	black	—
table_chrome	blue	black	—
table_header	cyan	black	yes
table_body	bright_white	black	—
disabled_row	bright_black	black	—
selected	black	bright_blue	—
alert	bright_white	red	yes
help_header	bright_blue	black	yes
help_panel	white	black	—
map_dot	green	black	—
map_crosshair	bright_red	black	yes
map_center	bright_white	black	yes
quote	white	black	—
quote_author	white	black	—
report_header	cyan	black	—
indicator_on	bright_green	black	yes
indicator_off	bright_black	black	—

Star decoration colors (6 slots, cycling):

star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"